



Tecnológico de Monterrey

Competencia Desarrollo de componentes de software

Axel Orlando Arenas Vergara A01278654

Construcción de software y toma de decisiones (Gpo 501)

Profesores:

Enrique Alfonso Calderón Balderas

Denisse Lizbeth Maldonado Flores

Alejandro Fernández Vilchis

Apuntes y conceptos aprendidos en clase

En esta sección se presentan algunos conceptos aprendidos durante las clases relacionados con el desarrollo de componentes de software. Estos temas me ayudaron a comprender cómo se construyen las diferentes partes de una página web, cómo se comunican entre sí y la importancia de mantener una estructura organizada durante el desarrollo de un proyecto.

Front-end

El front-end es la parte del sistema con la que interactúa directamente el usuario. Incluye las pantallas, formularios, tablas, botones y elementos visuales de la aplicación. Durante el curso entendí que no se trata únicamente de que una pantalla se vea bien, sino de que sea clara, fácil de usar y permita realizar las tareas de forma sencilla.

En el proyecto observé que el front-end funciona como el punto de comunicación entre el usuario y el sistema, ya que es donde se capturan los datos que posteriormente serán procesados por el servidor.

Back-end

El back-end es la parte encargada de procesar la información y ejecutar la lógica del sistema. Aquí se realizan validaciones, consultas a la base de datos y operaciones necesarias para que las funcionalidades funcionen correctamente. Aunque el usuario normalmente no ve esta parte, es fundamental para el funcionamiento de la aplicación.

En el proyecto pude observar que muchas acciones realizadas desde las pantallas dependían del back-end para almacenar información, consultar registros y devolver resultados que después se mostraban al usuario.

Cliente-Servidor

El modelo cliente-servidor consiste en la comunicación entre una aplicación utilizada por el usuario y un servidor encargado de procesar las solicitudes. El cliente envía información o realiza una petición, mientras que el servidor la recibe, la procesa y devuelve una respuesta.

Durante el desarrollo del proyecto entendí que gran parte del funcionamiento del sistema depende de esta comunicación constante, ya que cada consulta, registro o modificación de información requiere un intercambio de datos entre ambas partes.

API

Una API es un mecanismo que permite la comunicación entre diferentes componentes de software mediante solicitudes y respuestas. Su función es facilitar el intercambio de información sin que cada parte tenga que acceder directamente a los procesos internos de la otra.

En el proyecto observé que las APIs ayudan a conectar diferentes elementos del sistema, permitiendo que la información pueda enviarse, procesarse y recuperarse de manera organizada.

Convenciones de código

Las convenciones de código son un conjunto de reglas que ayudan a mantener la programación organizada y comprensible para todos los integrantes de un equipo. Incluyen aspectos como la estructura de carpetas, nombres de archivos, nombres de variables y la forma de escribir el código.

Durante el proyecto entendí que seguir convenciones facilita la lectura del código, reduce confusiones y permite que varias personas trabajen sobre un mismo sistema de forma más ordenada.

Pair Programming

El pair programming es una práctica en la que dos personas colaboran en una misma tarea de programación. Mientras una persona desarrolla una parte del código, la otra puede revisar la lógica, detectar errores o proponer mejoras.

En las actividades del curso y del proyecto observé que esta forma de trabajo ayuda a resolver problemas más rápidamente y favorece el intercambio de conocimientos entre los integrantes del equipo.

Aplicado en el proyecto

Componentes de front-end desarrollados

Durante el desarrollo del proyecto participé en la implementación y modificación de distintos componentes del frontend del sistema, principalmente en las vistas de Login, Dashboard, Clientes, Operaciones, Alertas y Reportes.

Durante las primeras etapas del proyecto colaboré en la construcción de las interfaces utilizando HTML, CSS y JavaScript. Participé en la organización de los elementos visuales de distintas pantallas, definiendo la estructura de formularios, tablas, contenedores y menús de navegación que posteriormente formarían parte

del sistema. En varias ocasiones realicé ajustes en archivos HTML para reorganizar elementos de la interfaz, mejorar la distribución de la información y facilitar la navegación entre módulos.

También participé en la implementación y validación de componentes relacionados con el inicio de sesión. Durante las pruebas revisé que el usuario pudiera autenticarse correctamente y que, una vez iniciada la sesión, fuera redirigido al Dashboard. Además, verifiqué que los usuarios sin autenticación fueran enviados nuevamente a la pantalla de Login para evitar accesos no autorizados.

Otra actividad importante fue la revisión de las tablas dinámicas utilizadas en los módulos de Clientes, Operaciones, Alertas y Reportes. Estas tablas se alimentaban con información proveniente de la base de datos mediante JavaScript. Durante las pruebas validé que los registros se mostraran correctamente, que la información coincidiera con la almacenada en la base de datos y que los componentes continuaran funcionando después de cada modificación realizada.

Posteriormente participé en la mejora visual de la aplicación. Durante las primeras versiones observé que varios componentes estaban desarrollados únicamente con CSS personalizado. Por esta razón propuse utilizar Bootstrap, framework que habíamos estudiado durante el curso. Esta propuesta permitió simplificar parte del código relacionado con estilos y utilizar componentes visuales más consistentes para tablas, formularios, botones y contenedores. Además, colaboré explicando a otros integrantes del equipo qué elementos podían modificarse dentro de los archivos HTML para aprovechar las clases proporcionadas por Bootstrap y reducir la cantidad de estilos personalizados necesarios.

Finalmente, durante la etapa de validación con el socio formador participé en la modificación de las tablas utilizadas para mostrar información proveniente de la base de datos. Después de recibir la retroalimentación correspondiente, revisé los archivos HTML y JavaScript de cada módulo para localizar dónde se generaban visualmente los identificadores y folios internos. Por ejemplo, en el módulo de Clientes eliminé la columna ``<th>ID</th>`` del archivo clientes.html.

```
<thead>
  <tr>
    <th>Nombre Completo</th>
    <th>RFC</th>
    <th>Riesgo</th>
    <th>Acciones</th>
  </tr>
</thead>
```

Para realizar esta corrección modifiqué directamente la estructura HTML de la tabla de clientes. El socio formador indicó que los identificadores internos no debían mostrarse al usuario final, por lo que eliminé la columna ID que aparecía en versiones anteriores del sistema.

Después de realizar el cambio verifiqué que la tabla continuara mostrando correctamente la información de los clientes y que la distribución visual no se viera afectada.

Fecha	Cliente	Contrato	Tipo	Monto	Acciones
25/4/2026	Juan Manuel González de Cossío	C-001	DEPÓSITO	\$5000.00	Ver Expediente Cliente
25/4/2026	Juan Manuel González de Cossío	C-001	RETIRO	\$2000.00	Ver Expediente Cliente
25/4/2026	María Rodríguez López	C-002	DEPÓSITO	\$8000.00	Ver Expediente Cliente
25/4/2026	María Rodríguez López	C-002	RETIRO	\$1500.00	Ver Expediente Cliente
25/4/2026	Juan Pérez Hernández	C-003	TRANSFERENCIA	\$12000.00	Ver Expediente Cliente
25/4/2026	Juan Pérez Hernández	C-003	RETIRO	\$3000.00	Ver Expediente Cliente
25/4/2026	Ana López García	C-004	DEPÓSITO	\$7000.00	Ver Expediente Cliente

Sistema sin folios

Posteriormente modifiqué el archivo `clientes.js`, eliminando la línea que construía el identificador del cliente dentro de la tabla mediante el código `C-00\${c.id}`. Después de realizar estos cambios también ajusté los valores de `colspan` para evitar que los mensajes de carga y error quedaran desalineados.

```
tr.innerHTML = `
    <td><strong>${c.nombre_completo}</strong></td>
    <td>${c.rfc}</td>
    <td>${etiquetaRiesgo}</td>
    <td>
        <button class="btn-ver" data-id="${c.id}">Ver Detalle</button>
    </td>
`
```

Además de modificar el HTML también tuve que actualizar el código JavaScript encargado de construir dinámicamente las filas de la tabla. Eliminé la línea que mostraba el identificador del cliente mediante la expresión C-00\${c.id}, ya que esta información ya no debía visualizarse.

Sin embargo, conservé el valor `data-id="{c.id}"` porque sigue siendo necesario internamente para identificar qué cliente debe abrirse cuando el usuario selecciona la opción de ver detalle.

Realicé modificaciones similares en los módulos de Operaciones, Alertas, Reportes y Dashboard, donde identifiqué y eliminé las líneas encargadas de mostrar folios e identificadores visibles para el usuario. Una vez realizados los cambios probé nuevamente cada vista para verificar que la eliminación de estas columnas no afectara la carga de datos ni la navegación entre módulos.

Sistema SVA

Dashboard

Cientes

Operaciones

Alertas

Reportes

Cerrar sesión

Dashboard Principal

Usuario: Oficial Titular
Rol: Oficial de Cumplimiento

Resumen general

Cientes
Total de clientes: 21

Alertas
Total de alertas pendientes: 0

Cientes recientes

Nombre Completo	RFC	Correo
Pablo Gomez Perez	GOMP900101XYZ	pablo.gomez@ejemplo.com
Fernanda Silva Gómez	SILF040202TTT	fernanda@gmail.com
Antonio Ortega Ruiz	ORTA030101SSS	antonio@gmail.com
Mariana Navarro Díaz	NAVM020909RRR	mariana@gmail.com
Sergio Lozano Cruz	LOZS010808QQQ	sergio@gmail.com

Alertas recientes

Cliente	Regla Rota	Estatus
Juan Manuel González de Cossío	Depósito inusual para su perfil	Investigando
Juan Manuel González de Cossío	Depósito inusual para su perfil	Investigando

Posteriormente incorporé funciones JavaScript encargadas de validar que el archivo seleccionado tuviera extensión `.csv`, habilitar el botón de carga y enviar el archivo completo al servidor mediante `FormData` y una petición HTTP con `fetch`. De esta forma, el backend podía recibir el archivo CSV y procesarlo para registrar la información correspondiente en la base de datos.

```

<div class="drop-zone" id="drop-zone-csv">
  <p>Arrastra tu archivo .csv aquí o <strong>haz clic
  para buscar</strong></p>
  <input type="file" id="input-file-csv" accept=".
  csv" style="display: none;">
</div>
<div id="archivo-seleccionado" style="margin-bottom:
15px; font-weight: bold;"></div>
<button type="submit" class="btn-primary"
id="btn-enviar-csv" disabled>Iniciar Ingesta Masiva</
button>

```

Para implementar la funcionalidad de carga masiva agregué una zona de carga de archivos dentro de la vista de Ingesta Masiva. Este componente permite que el usuario seleccione un archivo CSV desde su equipo o lo arrastre directamente sobre la interfaz.

```

function actualizarArchivoSeleccionado(file) {
  if (file && file.name.endsWith('.csv')) {
    archivoSeleccionadoTxt.textContent = `Archivo listo: ${file.name} (${
      (file.size/1024).toFixed(2)} KB)`;
    btnEnviar.disabled = false;
    mostrarMensaje('', false);
  } else {
    archivoSeleccionadoTxt.textContent = '';
    btnEnviar.disabled = true;
    mostrarMensaje('Error: Por favor selecciona un archivo con
    extensión .csv válido.', true);
  }
}

```

Desarrollé una validación que verifica que el archivo seleccionado tenga extensión .csv antes de permitir su envío al servidor.

Si el archivo cumple con el formato esperado, el sistema habilita automáticamente el botón de carga y muestra información básica del archivo seleccionado. En caso contrario, se muestra un mensaje de error y el proceso de carga permanece bloqueado.

Esta validación evita que se intenten procesar archivos incompatibles dentro del sistema.

Componentes de back-end desarrollados

Durante el proyecto participé en la revisión y validación de distintos componentes del backend encargados de procesar información y comunicarse con la base de datos.

Una de las actividades que realicé consistió en revisar el funcionamiento de las rutas utilizadas por el sistema para recuperar información de clientes, operaciones, alertas y reportes. Para ello ejecuté el sistema localmente y verifiqué que las peticiones realizadas desde el frontend llegaran correctamente a las rutas definidas dentro del backend. Cuando una vista no mostraba información o presentaba errores, revisaba la consola del navegador y posteriormente analizaba las rutas y controladores correspondientes para identificar el origen del problema.

También participé en la validación del flujo utilizado para registrar clientes. Antes de implementar la importación mediante archivos CSV revisé cómo funcionaba el registro tradicional de clientes a través de la ruta `/clientes`. Esto me permitió comprender que el controlador ya contenía la lógica necesaria para validar la información recibida y almacenarla en la base de datos. A partir de este análisis aproveché la misma ruta existente para registrar múltiples clientes provenientes de un archivo CSV sin necesidad de desarrollar un nuevo endpoint.

Otra actividad importante fue la revisión del comportamiento del servidor durante distintas pruebas de ejecución. En varias ocasiones aparecieron errores relacionados con dependencias faltantes, configuración del entorno y conexión con la base de datos. Para validar que el backend estuviera funcionando correctamente ejecuté el servidor localmente y comprobé que respondiera adecuadamente a las peticiones realizadas desde las diferentes vistas del sistema.

```
1  import express from 'express';
2  import { registrarCliente, listarClientes } from '../controllers/
    clienteController.js';
3  import { verificarToken } from '../middleware/authMiddleware.js'; // Importamos
    el candado
4
5  const router = express.Router();
6
7  // Verificación de token
8  router.get('/', verificarToken, listarClientes);
9  router.post('/', verificarToken, registrarCliente);
10
11 export default router;
```

La captura muestra una de las rutas utilizadas por el sistema para gestionar clientes. Durante mis revisiones identifiqué que la ruta GET permitía recuperar los registros

almacenados en la base de datos, mientras que la ruta POST era utilizada para registrar nuevos clientes.

Interacción cliente-servidor

Una parte importante de mi aprendizaje durante el proyecto fue comprender cómo se comunican los componentes del frontend con el backend y la base de datos. Antes de este proyecto conocía el concepto de cliente-servidor de forma teórica, pero fue durante el desarrollo cuando pude observar y validar el flujo completo de la información.

Al trabajar con módulos como Clientes, Operaciones, Alertas y Reportes revisé cómo cada pantalla obtenía información desde el servidor. Por ejemplo, cuando el usuario ingresaba al módulo de Clientes, el archivo JavaScript correspondiente ejecutaba una petición utilizando la función `peticionProtegida('/clientes')`. Esta petición era enviada al backend, donde la ruta correspondiente recibía la solicitud, ejecutaba el controlador asociado y posteriormente realizaba una consulta a la base de datos. Finalmente, la información obtenida era enviada nuevamente al frontend en formato JSON para construir dinámicamente la tabla mostrada al usuario.

En este fragmento muestro cómo el Dashboard obtiene información de tres servicios diferentes del backend: `/dashboard`, `/clientes` y `/alertas`. Utilicé `Promise.all` para que las peticiones se ejecutaran al mismo tiempo y la pantalla cargara más rápido. Esta parte evidencia la comunicación cliente-servidor porque el frontend solicita datos al backend y después utiliza la respuesta para construir las tarjetas y tablas del Dashboard.

```
const [resumen, clientes, alertas] = await Promise.all([
  peticionProtegida('/dashboard'),
  peticionProtegida('/clientes'),
  peticionProtegida('/alertas')
]);
```

Durante las pruebas participé verificando que la información mostrada en las vistas coincidiera con los datos almacenados en la base de datos. Cuando alguna tabla no mostraba registros o presentaba comportamientos inesperados, revisaba si el problema se encontraba en la petición enviada desde el frontend, en la respuesta generada por el backend o en la consulta realizada a la base de datos.

También pude observar este flujo durante la implementación de la funcionalidad de importación mediante archivos CSV. Después de leer el archivo utilizando JavaScript, cada registro era enviado al servidor utilizando la misma ruta utilizada

para registrar clientes individualmente. El backend recibía la información, ejecutaba las validaciones correspondientes y almacenaba los registros en la base de datos. Una vez completado el proceso, el frontend realizaba nuevamente la consulta para actualizar la tabla y mostrar los nuevos clientes registrados.

Otra situación en la que participé fue durante el despliegue del sistema. Inicialmente la aplicación realizaba las peticiones utilizando una dirección local (`localhost:3000`), lo cual funcionaba únicamente en el entorno de desarrollo. Posteriormente fue necesario actualizar la configuración para utilizar la dirección pública del servidor desplegado, permitiendo que otros usuarios pudieran acceder al sistema sin necesidad de ejecutarlo localmente. Esta actividad me permitió comprender mejor la importancia de la comunicación entre cliente y servidor dentro de una aplicación web.

Servicios web utilizados

Durante el desarrollo del proyecto utilicé distintos servicios web para permitir la comunicación entre la interfaz del sistema, el servidor y la base de datos. Estos servicios fueron implementados mediante endpoints que recibían solicitudes desde el frontend y devolvían la información necesaria para construir las diferentes vistas del sistema.

Uno de los servicios que más utilicé fue el endpoint `/clientes`, el cual permitía consultar la información almacenada de los clientes y mostrarla dentro del módulo correspondiente. Este mismo endpoint fue utilizado posteriormente durante la implementación de la funcionalidad de importación mediante archivos CSV, ya que los registros obtenidos del archivo eran enviados al servidor utilizando solicitudes HTTP para posteriormente almacenarse en la base de datos.

También participé en pruebas relacionadas con los endpoints `/operaciones`, `/alertas`, `/reportes` y `/dashboard`, los cuales permitían recuperar información necesaria para construir las tablas y paneles visuales del sistema. Cada uno de estos servicios obtenía información desde la base de datos y la enviaba al frontend en formato JSON para que pudiera ser procesada mediante JavaScript.

Adicionalmente se utilizó un servicio de autenticación que permitía validar usuarios y generar el token utilizado para controlar el acceso a las diferentes pantallas del sistema. Durante las pruebas verifiqué que los usuarios sin sesión activa fueran redirigidos al login y que únicamente los usuarios autenticados pudieran acceder a los módulos protegidos.

Pair Programming

Durante el desarrollo del proyecto realicé diversas actividades de programación colaborativa con mis compañeros de equipo para resolver problemas técnicos y validar implementaciones.

Por ejemplo, trabajé junto con mi compañero Iker para resolver errores relacionados con funcionalidades del backend y configuración del sistema. Cuando encontraba algún problema cuya causa no lograba identificar fácilmente, revisábamos juntos el código para localizar el origen del error y determinar qué archivo debía modificarse. Esto ocurrió especialmente durante las pruebas de despliegue del sistema, donde inicialmente la aplicación estaba configurada para comunicarse con un servidor local mediante la dirección `http://localhost:3000`. Durante la revisión conjunta identificamos que esta configuración impediría que los profesores utilizaran el sistema desde Internet. Como resultado, modificamos la configuración para utilizar la dirección pública desplegada en Render mediante la variable:

```
const API_BASE_URL = 'https://financial-sentinel.onrender.com/api';
```

Después de realizar este cambio validamos nuevamente la comunicación entre frontend y backend para confirmar que las peticiones continuaran funcionando correctamente.

También realicé actividades de programación colaborativa con mi compañera Isabella durante la mejora visual de la aplicación. Mientras trabajábamos en la interfaz propuse utilizar Bootstrap, framework que habíamos estudiado durante el curso, debido a que la versión inicial construida únicamente con CSS requería una gran cantidad de estilos personalizados para mantener una apariencia uniforme. Durante esta actividad revisamos juntos los archivos HTML y CSS para identificar componentes que podían beneficiarse del uso de clases proporcionadas por Bootstrap. Además, expliqué qué elementos debían modificarse dentro de las vistas para aprovechar las utilidades del framework y reducir parte del código CSS que se estaba utilizando para estilos básicos de presentación.

Reflexión

Durante el desarrollo de esta competencia comprendí mejor cómo se construye una aplicación web a partir de diferentes componentes que trabajan de manera conjunta. Antes conocía conceptos como front-end, back-end o cliente-servidor principalmente desde la teoría, pero fue durante el proyecto cuando pude observar cómo cada uno de ellos participa en el funcionamiento completo del sistema.

También aprendí que desarrollar software no consiste únicamente en escribir código. Gran parte del trabajo implica comprender cómo se comunican las diferentes partes de la aplicación, identificar errores, realizar pruebas y mantener una estructura organizada que facilite el trabajo en equipo. Actividades como revisar rutas, validar información, modificar interfaces o analizar la comunicación entre el frontend y el backend me ayudaron a entender mejor este proceso.

Otro aspecto importante fue el trabajo colaborativo. A través de actividades de pair programming y revisiones conjuntas con mis compañeros pude observar que muchas veces una segunda perspectiva ayuda a detectar problemas o encontrar soluciones que individualmente podrían pasar desapercibidas. Además, seguir convenciones de código y mantener una organización adecuada facilitó la integración de los distintos componentes desarrollados durante el proyecto.

Considero que esta competencia fortaleció mi comprensión sobre el desarrollo de software, ya que me permitió participar en tareas relacionadas tanto con la interfaz del usuario como con la lógica interna del sistema. Gracias a esta experiencia ahora tengo una visión más completa de cómo se construye, integra y mantiene una aplicación web funcional.